

GUIDA TECNICA

Integrazione di una Nuova Tecnologia di Batteria nel Sistema BESS

Versione: 3.8.0

Autore: Lorenzo Giannuzzo

Affiliazione: Politecnico di Torino, DENERG, Energy Center Lab

INTRODUZIONE

Questa guida descrive la procedura completa per integrare una nuova tecnologia di batteria nel sistema di ottimizzazione BESS. L'architettura del codice è stata progettata per supportare l'estensione a nuove tecnologie attraverso un approccio modulare. L'integrazione richiede modifiche in sei aree principali del codice, che verranno descritte in dettaglio nelle sezioni seguenti.

Per questa guida utilizzeremo come esempio l'integrazione di una batteria al Sodio-Ione (Na-Ion), tecnologia emergente con caratteristiche intermedie tra litio-ione e grafene.

SEZIONE 1: DEFINIZIONE DEI PARAMETRI GLOBALI DELLA TECNOLOGIA

La prima operazione consiste nella definizione dei parametri operativi specifici della nuova tecnologia. Questi parametri devono essere inseriti nella Sezione 1 del codice, immediatamente dopo i parametri delle tecnologie esistenti (circa riga 90-110).

Individuare il blocco di codice contenente i parametri per litio-ione e grafene:

```
LITHIUM_ION_SOC_MIN = 0.1
LITHIUM_ION_SOC_MAX = 0.9
LITHIUM_ION_DOD = 0.8
LITHIUM_ION_EOL_CYCLES = 6000

GRAPHENE_SOC_MIN = 0.0
GRAPHENE_SOC_MAX = 1.0
GRAPHENE_DOD = 1.0
GRAPHENE_EOL_CYCLES = 500000
```

Aggiungere immediatamente dopo un nuovo blocco per la tecnologia Sodio-Ione:

```
SODIUM_ION_SOC_MIN = 0.05
SODIUM_ION_SOC_MAX = 0.95
SODIUM_ION_DOD = 0.90
SODIUM_ION_EOL_CYCLES = 4000
```

Descrizione dei parametri:

SOC_MIN rappresenta il livello minimo di State of Charge ammissibile per preservare la vita della batteria. Per il sodio-ione questo valore è tipicamente inferiore rispetto al litio-ione grazie alla maggiore tolleranza della chimica agli stati di carica bassi.

SOC_MAX rappresenta il livello massimo di State of Charge ammissibile. Per il sodio-ione si può arrivare al 95% senza stress eccessivo sugli elettrodi.

DOD (Depth of Discharge) rappresenta la profondità di scarica utilizzabile, calcolata come differenza tra SOC_MAX e SOC_MIN. Per il sodio-ione questo valore è 0.90, superiore al litio-ione.

EOL_CYCLES indica il numero di cicli equivalenti prima che la capacità scenda sotto l'80% del valore nominale. Il sodio-ione ha tipicamente una vita ciclica leggermente inferiore al litio-ione ma con costi per ciclo competitivi.

SEZIONE 2: INSERIMENTO DEI DATI SPERIMENTALI DI EFFICIENZA

I dati sperimentali di efficienza devono essere inseriti nella Sezione 2 del codice, dopo i dizionari esistenti per litio-ione e grafene (circa riga 130-160).

I dati sono organizzati in dizionari separati per ogni C-rate testato (tipicamente 0.5C e 1C). Ogni dizionario contiene quattro array con i risultati delle prove sperimentali ripetute.

Individuare i dizionari esistenti:

```
LITHIUM_ION_05C_DATA = { ... }
LITHIUM_ION_1C_DATA = { ... }
GRAPHENE_05C_DATA = { ... }
GRAPHENE_1C_DATA = { ... }
```

Aggiungere i dizionari per la nuova tecnologia:

```
SODIUM_ION_05C_DATA = {
    'charge_energy_kwh': [11.52, 11.48, 11.55],
    'discharge_energy_kwh': [10.72, 10.68, 10.75],
    'energy_efficiency': [0.930, 0.928, 0.932],
    'coulombic_efficiency': [0.992, 0.990, 0.993]
}

SODIUM_ION_1C_DATA = {
    'charge_energy_kwh': [11.45, 11.50, 11.42],
    'discharge_energy_kwh': [10.35, 10.42, 10.38],
    'energy_efficiency': [0.904, 0.906, 0.908],
    'coulombic_efficiency': [0.978, 0.980, 0.976]
}
```

Descrizione dei campi:

charge_energy_kwh contiene l'energia assorbita durante la fase di carica per ogni test ripetuto, espressa in kWh. Questi valori derivano tipicamente da prove di laboratorio su celle o moduli di riferimento.

discharge_energy_kwh contiene l'energia erogata durante la fase di scarica per ogni test ripetuto. La differenza rispetto all'energia di carica rappresenta le perdite del ciclo.

energy_efficiency contiene l'efficienza di round-trip calcolata come rapporto tra energia scaricata ed energia caricata per ogni test. Questo è il parametro principale utilizzato dall'ottimizzatore.

coulombic_efficiency contiene l'efficienza coulombica, ovvero il rapporto tra carica elettrica estratta e carica immessa. Valori inferiori a 1 indicano reazioni parassite o perdite di capacità.

Per ottenere dati sperimentali affidabili è necessario eseguire prove cicliche su celle rappresentative della tecnologia, seguendo protocolli standardizzati come quelli definiti da IEC 62620 o protocolli interni di laboratorio. In assenza di dati sperimentali propri, è possibile utilizzare valori dalla letteratura tecnica o dai datasheet dei produttori, verificandone la coerenza con le condizioni operative previste.

SEZIONE 3: MODIFICA DELLA CLASSE BatteryEfficiencyModel

La classe BatteryEfficiencyModel gestisce la selezione dei dati sperimentali e il calcolo delle efficienze operative. La modifica richiede l'aggiunta di un nuovo ramo condizionale nel costruttore della classe (circa riga 180-220).

Individuare il blocco if-elif nel metodo `__init__`:

```
if technology == "LITIO-IONE":
    if c_rate <= 0.5:
        self.data = LITHIUM_ION_05C_DATA
    else:
        self.data = LITHIUM_ION_1C_DATA
elif technology == "GRAFENE":
    if c_rate <= 0.5:
        self.data = GRAPHENE_05C_DATA
    else:
        self.data = GRAPHENE_1C_DATA
else:
    raise ValueError(f"Tecnologia non supportata: {technology}")
```

Modificare aggiungendo il ramo per la nuova tecnologia prima del blocco else:

```
if technology == "LITIO-IONE":
    if c_rate <= 0.5:
        self.data = LITHIUM_ION_05C_DATA
    else:
        self.data = LITHIUM_ION_1C_DATA
elif technology == "GRAFENE":
    if c_rate <= 0.5:
        self.data = GRAPHENE_05C_DATA
    else:
        self.data = GRAPHENE_1C_DATA
elif technology == "SODIO-IONE":
    if c_rate <= 0.5:
        self.data = SODIUM_ION_05C_DATA
    else:
        self.data = SODIUM_ION_1C_DATA
else:
    raise ValueError(f"Tecnologia non supportata: {technology}")
```

La logica di selezione basata sul C-rate permette di utilizzare dati di efficienza appropriati per le condizioni operative. Se la tecnologia presenta comportamenti significativamente diversi a C-rate intermedi, è possibile estendere la logica con soglie aggiuntive.

SEZIONE 4: MODIFICA DELLA CLASSE Battery

La classe Battery rappresenta il modello fisico della batteria e deve essere modificata per riconoscere la nuova tecnologia e assegnarle i parametri corretti. La modifica riguarda il metodo `__init__` (circa riga 350-400).

Individuare il blocco if-elif che assegna i parametri in base alla tecnologia:

```
if technology == "LITIO-IONE":
    self.soc_min = LITHIUM_ION_SOC_MIN
    self.soc_max = LITHIUM_ION_SOC_MAX
    self.dod = LITHIUM_ION_DOD
    self.eol_cycles = LITHIUM_ION_EOL_CYCLES
elif technology == "GRAFENE":
    self.soc_min = GRAPHENE_SOC_MIN
    self.soc_max = GRAPHENE_SOC_MAX
    self.dod = GRAPHENE_DOD
    self.eol_cycles = GRAPHENE_EOL_CYCLES
else:
    raise ValueError(f"Tecnologia non supportata: {technology}")
```

Aggiungere il ramo per la nuova tecnologia:

```
if technology == "LITIO-IONE":
    self.soc_min = LITHIUM_ION_SOC_MIN
    self.soc_max = LITHIUM_ION_SOC_MAX
    self.dod = LITHIUM_ION_DOD
    self.eol_cycles = LITHIUM_ION_EOL_CYCLES
elif technology == "GRAFENE":
    self.soc_min = GRAPHENE_SOC_MIN
    self.soc_max = GRAPHENE_SOC_MAX
    self.dod = GRAPHENE_DOD
    self.eol_cycles = GRAPHENE_EOL_CYCLES
elif technology == "SODIO-IONE":
    self.soc_min = SODIUM_ION_SOC_MIN
    self.soc_max = SODIUM_ION_SOC_MAX
    self.dod = SODIUM_ION_DOD
    self.eol_cycles = SODIUM_ION_EOL_CYCLES
else:
    raise ValueError(f"Tecnologia non supportata: {technology}")
```

5.2 Modifica del metodo update_degradation nella classe Battery

Il metodo `update_degradation` deve essere modificato per chiamare la funzione di degrado appropriata in base alla tecnologia.

Individuare il metodo esistente (circa riga 450):

```
def update_degradation(self):
    if self.technology == "LITIO-IONE":
        self.equivalent_cycles = self.throughput_kwh / (2 * 10 * self.nominal_capacity * 1000)
        capacity_percentage = degradation(self.equivalent_cycles)
        self.capacity = self.nominal_capacity * (capacity_percentage / 100.0)
        if MACSE_ENABLED:
            macse_percentage_original = MACSE_CAPACITY_MWH / self.nominal_capacity
            self.macse_capacity = self.nominal_capacity * macse_percentage_original
            self.trading_capacity = self.capacity - self.macse_capacity
    else:
        self.capacity = self.nominal_capacity
```

Modificare per includere la nuova tecnologia:

```
def update_degradation(self):
    if self.technology == "LITIO-IONE":
        self.equivalent_cycles = self.throughput_kwh / (2 * self.nominal_capacity * 1000 * self.dod)
        capacity_percentage = degradation(self.equivalent_cycles)
        self.capacity = self.nominal_capacity * (capacity_percentage / 100.0)
        if MACSE_ENABLED:
            macse_percentage_original = MACSE_CAPACITY_MWH / self.nominal_capacity
            self.macse_capacity = self.nominal_capacity * macse_percentage_original
            self.trading_capacity = self.capacity - self.macse_capacity

    elif self.technology == "SODIO-IONE":
        self.equivalent_cycles = self.throughput_kwh / (2 * self.nominal_capacity * 1000 * self.dod)
        capacity_percentage = degradation_sodium_ion(self.equivalent_cycles)
        self.capacity = self.nominal_capacity * (capacity_percentage / 100.0)
        if MACSE_ENABLED:
            macse_percentage_original = MACSE_CAPACITY_MWH / self.nominal_capacity
            self.macse_capacity = self.nominal_capacity * macse_percentage_original
            self.trading_capacity = self.capacity - self.macse_capacity

    elif self.technology == "GRAFENE":
        # Grafene: degrado trascurabile, modello lineare semplificato
        self.equivalent_cycles = self.throughput_kwh / (2 * self.nominal_capacity * 1000 * self.dod)
        # Perdita minima: 0.00002% per ciclo
        capacity_percentage = max(80, 100 - (0.00002 * self.equivalent_cycles))
        self.capacity = self.nominal_capacity * (capacity_percentage / 100.0)

    else:
        self.capacity = self.nominal_capacity
```


SEZIONE 6: MODIFICA DEL PARSER DEGLI ARGOMENTI

Il parser degli argomenti da linea di comando deve essere aggiornato per riconoscere la nuova tecnologia come opzione valida.

Individuare la definizione dell'argomento battery-tech nella funzione parse_arguments (circa riga 55):

```
battery_group.add_argument('--battery-tech', type=str,
                           choices=['LITIO-IONE', 'GRAFENE'],
                           default='LITIO-IONE',
                           help='Tecnologia batteria (default: LITIO-IONE)')
```

Modificare aggiungendo la nuova tecnologia alle scelte disponibili:

```
battery_group.add_argument('--battery-tech', type=str,
                           choices=['LITIO-IONE', 'GRAFENE', 'SODIO-IONE'],
                           default='LITIO-IONE',
                           help='Tecnologia batteria (default: LITIO-IONE)')
```

Aggiungere inoltre i parametri SOC specifici per la nuova tecnologia nel gruppo soc_group:

```
soc_group.add_argument('--sodium-soc-min', type=float, default=0.05,
                       help='SOC minimo sodio-ione (default: 0.05)')
soc_group.add_argument('--sodium-soc-max', type=float, default=0.95,
                       help='SOC massimo sodio-ione (default: 0.95)')
```

Aggiungere la validazione dei nuovi parametri nella sezione di validazione:

```
if not (0 <= args.sodium_soc_min < args.sodium_soc_max <= 1):
    parser.error(f"SOC sodio-ione invalido: min={args.sodium_soc_min}, max={args.sodium_soc_max}")
```

SEZIONE 7: AGGIORNAMENTO DELLE VARIABILI GLOBALI NELLA FUNZIONE MAIN

La funzione main deve essere aggiornata per leggere i parametri SOC dalla linea di comando e applicarli alle variabili globali.

Individuare il blocco di override delle variabili globali nella funzione main (circa riga 1850):

```
global POD_POWER_MW, BATTERY_TECHNOLOGY, BATTERY_CAPACITY_MWH, ...

LITHIUM_ION_SOC_MIN = args.lithium_soc_min
LITHIUM_ION_SOC_MAX = args.lithium_soc_max
GRAPHENE_SOC_MIN = args.graphene_soc_min
GRAPHENE_SOC_MAX = args.graphene_soc_max
```

Aggiungere le variabili per la nuova tecnologia:

```
global POD_POWER_MW, BATTERY_TECHNOLOGY, BATTERY_CAPACITY_MWH, ...
global SODIUM_ION_SOC_MIN, SODIUM_ION_SOC_MAX

LITHIUM_ION_SOC_MIN = args.lithium_soc_min
LITHIUM_ION_SOC_MAX = args.lithium_soc_max
GRAPHENE_SOC_MIN = args.graphene_soc_min
GRAPHENE_SOC_MAX = args.graphene_soc_max
SODIUM_ION_SOC_MIN = args.sodium_soc_min
SODIUM_ION_SOC_MAX = args.sodium_soc_max
```

Aggiornare anche la stampa della configurazione per mostrare il range SOC corretto:

```
if args.battery_tech == "LITIO-IONE":
    print(f" SOC range:      {args.lithium_soc_min * 100:.0f}% - {args.lithium_soc_max * 100:.0f}%")
elif args.battery_tech == "GRAFENE":
    print(f" SOC range:      {args.graphene_soc_min * 100:.0f}% - {args.graphene_soc_max * 100:.0f}%")
elif args.battery_tech == "SODIO-IONE":
    print(f" SOC range:      {args.sodium_soc_min * 100:.0f}% - {args.sodium_soc_max * 100:.0f}%")
```

SEZIONE 8: VERIFICA E TEST

Dopo aver completato tutte le modifiche, è necessario verificare il corretto funzionamento attraverso una serie di test.

8.1 Test di base

Eseguire una simulazione minima per verificare che la nuova tecnologia sia riconosciuta:

```
python script.py --price-sell data/vendita.xlsx --price-buy data/acquisto.xlsx \
    --battery-tech SODIO-IONE
```

L'output dovrebbe mostrare la tecnologia selezionata nei messaggi di configurazione iniziale.

8.2 Test dei parametri SOC

Verificare che i limiti SOC vengano applicati correttamente:

```
python script.py --price-sell data/vendita.xlsx --price-buy data/acquisto.xlsx \
    --battery-tech SODIO-IONE --sodium-soc-min 0.10 --sodium-soc-max 0.90
```

Controllare nei risultati che il SOC non scenda mai sotto il 10% e non superi mai il 90%.

8.3 Test dell'efficienza

Eseguire una simulazione completa e verificare nei risultati che l'efficienza di round-trip corrisponda ai valori attesi dalla tabella dei dati sperimentali (circa 93% a 0.5C, 90% a 1C per il sodio-ione).

8.4 Test del degrado

Eseguire una simulazione su un periodo esteso (almeno 8760 ore per un anno) e verificare che la curva di SOH segua l'andamento previsto dal modello di degrado implementato.

8.5 Test di confronto

Eseguire simulazioni parallele con le tre tecnologie sugli stessi dati di input per verificare che i risultati economici riflettano le differenti caratteristiche tecniche:

```
python script.py --price-sell data/vendita.xlsx --price-buy data/acquisto.xlsx \
    --battery-tech LITIO-IONE --output-dir results/litio
```

```
python script.py --price-sell data/vendita.xlsx --price-buy data/acquisto.xlsx \
    --battery-tech GRAFENE --output-dir results/grafene
```

```
python script.py --price-sell data/vendita.xlsx --price-buy data/acquisto.xlsx \
    --battery-tech SODIO-IONE --output-dir results/sodio
```

Confrontare i profitti finali, i cicli equivalenti, e il SOH finale tra le tre tecnologie.

SEZIONE 9: DOCUMENTAZIONE E MANUTENZIONE

9.1 Documentazione del codice

Aggiungere commenti descrittivi per ogni nuova sezione di codice, indicando la fonte dei dati sperimentali e le assunzioni del modello di degrado.

9.2 Aggiornamento della documentazione utente

Aggiornare eventuali manuali utente o file README per includere la nuova tecnologia tra le opzioni disponibili, con una breve descrizione delle sue caratteristiche e dei casi d'uso appropriati.

9.3 Versionamento

Incrementare il numero di versione del codice e documentare le modifiche nel changelog, specificando la nuova tecnologia aggiunta e i parametri utilizzati.

APPENDICE A: RIEPILOGO DELLE MODIFICHE

File: script.py

Riga circa	Modifica
90-110	Aggiunta parametri globali SODIUM_ION_*
130-160	Aggiunta dizionari SODIUM_ION_05C_DATA e SODIUM_ION_1C_DATA
180-220	Modifica classe BatteryEfficiencyModel, aggiunta ramo elif
280-300	Aggiunta funzione degradation_sodium_ion
350-400	Modifica classe Battery.__init__, aggiunta ramo elif
450-480	Modifica metodo Battery.update_degradation, aggiunta ramo elif
55-60	Modifica parser, aggiunta tecnologia a choices
70-75	Aggiunta argomenti --sodium-soc-min e --sodium-soc-max
1850-1900	Modifica funzione main, aggiunta variabili globali e stampa

APPENDICE B: PARAMETRI TIPICI PER TECNOLOGIE COMUNI

Per riferimento, si riportano i parametri tipici di alcune tecnologie di accumulo che potrebbero essere integrate nel sistema:

Litio Ferro Fosfato (LFP):

SOC_MIN = 0.10, SOC_MAX = 0.95, DOD = 0.85, EOL_CYCLES = 4000-6000
Efficienza 0.5C = 94-96%, Efficienza 1C = 91-93%

Litio Nichel Manganese Cobalto (NMC):

SOC_MIN = 0.10, SOC_MAX = 0.90, DOD = 0.80, EOL_CYCLES = 2000-4000
Efficienza 0.5C = 94-96%, Efficienza 1C = 90-93%

Sodio-Ione (Na-Ion):

SOC_MIN = 0.05, SOC_MAX = 0.95, DOD = 0.90, EOL_CYCLES = 3000-5000
Efficienza 0.5C = 92-94%, Efficienza 1C = 88-91%

Flusso Vanadio (VRFB):

SOC_MIN = 0.15, SOC_MAX = 0.85, DOD = 0.70, EOL_CYCLES = 15000-20000
Efficienza 0.5C = 75-80%, Efficienza 1C = 70-75%

I valori riportati sono indicativi e possono variare significativamente in base al produttore, alla configurazione del sistema, e alle condizioni operative.

APPENDICE C: STRUTTURA DEI DATI SPERIMENTALI

Il formato dei dizionari dei dati sperimentali deve rispettare la seguente struttura:

```
{  
  'charge_energy_kwh': [valore_test1, valore_test2, valore_test3],  
  'discharge_energy_kwh': [valore_test1, valore_test2, valore_test3],  
  'energy_efficiency': [valore_test1, valore_test2, valore_test3],  
  'coulombic_efficiency': [valore_test1, valore_test2, valore_test3]  
}
```

I tre valori in ogni array rappresentano ripetizioni della stessa prova per valutare la variabilità sperimentale. Il sistema calcola automaticamente la media dei valori per ottenere l'efficienza operativa utilizzata nelle simulazioni.

Se sono disponibili più di tre ripetizioni, è possibile includere tutti i valori negli array. Se è disponibile un solo valore, replicarlo tre volte per mantenere la compatibilità con la struttura esistente.